



Convolutional neural network for image recognition

Dataset: The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems. The database is also widely used for training and testing in the field of machine learning.

Import libraries

```
In [1]: import torch
```

```
In [2]: from torchvision import datasets
from torchvision.transforms import ToTensor
train_data = datasets.MNIST(
    root = 'data',
    train = True,
    transform = ToTensor(),
    download = True,
)
test_data = datasets.MNIST(
    root = 'data',
    train = False,
    transform = ToTensor()
)
```

```
In [3]: print(train_data)
```

```
Dataset MNIST
  Number of datapoints: 60000
  Root location: data
  Split: Train
  StandardTransform
Transform: ToTensor()
```

```
In [4]: print(test_data)
```

```
Dataset MNIST
  Number of datapoints: 10000
  Root location: data
  Split: Test
  StandardTransform
Transform: ToTensor()
```

```
In [5]: print(train_data.data.size())
```

```
torch.Size([60000, 28, 28])
```

Visualization of MNIST dataset

Plot one train_data

```
In [6]: # Question: visualise one train_data
```

Plot multiple train_data

```
In [7]: # Question: visualise multiple train_data
```

Preparing data for training with DataLoaders

The Dataset retrieves our dataset's features and labels one sample at a time. While training a model, we typically want to pass samples in "minibatches", reshuffle the data at every epoch to reduce model overfitting, and use Python's multiprocessing to speed up data retrieval. DataLoader is an iterable that abstracts this complexity for us in an easy API.

```
In [8]: from torch.utils.data import DataLoader
loaders = {
    'train' : torch.utils.data.DataLoader(train_data,
                                          batch_size=100,
                                          shuffle=True,
                                          num_workers=1),

    'test'  : torch.utils.data.DataLoader(test_data,
                                          batch_size=100,
                                          shuffle=True,
                                          num_workers=1),
}
loaders
```

```
Out[8]: {'train': <torch.utils.data.dataloader.DataLoader at 0x106878510>,
        'test': <torch.utils.data.dataloader.DataLoader at 0x11fb68510>}
```

Define the Convolutional Neural Network model

A Convolutional Neural Network is type of neural network that is used mainly in image processing applications. Let us create convolution neural network using torch.nn.Module. torch.nn.Module will be base class for all neural network modules. We will use 2 fully convolutional layers, Relu activation function and MaxPooling.

```
In [9]: import torch.nn as nn
class CNN(nn.Module):
    def __init__(self):
```

```

super(CNN, self).__init__()
self.conv1 = nn.Sequential(
    nn.Conv2d(
        in_channels=1,
        out_channels=16,
        kernel_size=5,
        stride=1,
        padding=2,
    ),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2),
)
self.conv2 = nn.Sequential(
    nn.Conv2d(16, 32, 5, 1, 2),
    nn.ReLU(),
    nn.MaxPool2d(2),
)
# Question: create a fully connected layer, output 10 classes

def forward(self, x):
    x = self.conv1(x)
    x = self.conv2(x)
    # flatten the output of conv2 to (batch_size, 32 * 7 * 7)
    x = x.view(x.size(0), -1)
    output = self.out(x)
    return output, x    # return x for visualization

```

in_channels=1: because our input is a grayscale image.

Stride: is the number of pixels to pass at a time when sliding the convolutional kernel.

Padding: to preserve exactly the size of the input image, it is useful to add a zero padding on the border of the image.

kernel_size: we need to define a kernel which is a small matrix of size 5 * 5. To perform the convolution operation, we just need to slide the kernel along the image horizontally and vertically and do the dot product of the kernel and the small portion of the image.

The forward() pass defines the way we compute our output using the given layers and functions.

```

In [10]: cnn = CNN()
print(cnn)

```

```

CNN(
  (conv1): Sequential(
    (0): Conv2d(1, 16, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (conv2): Sequential(
    (0): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
)

```

```

In [11]: loss_func = nn.CrossEntropyLoss()
         loss_func

```

```

Out[11]: CrossEntropyLoss()

```

```

In [12]: from torch import optim
         optimizer = optim.Adam(cnn.parameters(), lr = 0.01)
         optimizer

```

```

Out[12]: Adam (
  Parameter Group 0
    amsgrad: False
    betas: (0.9, 0.999)
    capturable: False
    differentiable: False
    eps: 1e-08
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    weight_decay: 0
)

```

Train the model

Create a function called `train()` and pass num of epochs, model and data loaders as input parameters. `num_epochs`: Number of times our model will go through the entire training dataset

```

In [13]: from torch.autograd import Variable
         num_epochs = 10
         def train(num_epochs, cnn, loaders):

             cnn.train()

             # Train the model
             total_step = len(loaders['train'])

```

```

for epoch in range(num_epochs):
    for i, (images, labels) in enumerate(loaders['train']):

        # gives batch data, normalize x when iterate train_loader
        b_x = Variable(images) # batch x
        b_y = Variable(labels) # batch y
        output = cnn(b_x)[0]
        loss = loss_func(output, b_y)

        # Question: clear gradients for this training step

        # Question: backpropagation, compute gradients

        # Question: apply gradients

        if (i+1) % 100 == 0:
            print ('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
                  .format(epoch + 1, num_epochs, i + 1, total_step, loss.

        pass

    pass

pass
train(num_epochs, cnn, loaders)

```

```

-----
AttributeError                                Traceback (most recent call last)
Cell In[13], line 36
     32         pass
     35         pass
--> 36 train(num_epochs, cnn, loaders)

Cell In[13], line 16, in train(num_epochs, cnn, loaders)
     14 b_x = Variable(images)    # batch x
     15 b_y = Variable(labels)    # batch y
--> 16 output = cnn(b_x)[0]
     17 loss = loss_func(output, b_y)
     19 # Question: clear gradients for this training step
     20
     21
     22 # Question: backpropagation, compute gradients
     23
     24 # Question: apply gradients

File /Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/torch/nn/modules/module.py:1501, in Module._call_impl(self, *args, **kwargs)
    1496 # If we don't have any hooks, we want to skip the rest of the logic in
    1497 # this function, and just call forward.
    1498 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1499         or _global_backward_pre_hooks or _global_backward_hooks
    1500         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1501     return forward_call(*args, **kwargs)
    1502 # Do not call functions when jit is used
    1503 full_backward_hooks, non_full_backward_hooks = [], []

Cell In[9], line 28, in CNN.forward(self, x)
     26 # flatten the output of conv2 to (batch_size, 32 * 7 * 7)
     27 x = x.view(x.size(0), -1)
--> 28 output = self.out(x)
     29 return output, x

File /Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/torch/nn/modules/module.py:1614, in Module.__getattr__(self, name)
    1612     if name in modules:
    1613         return modules[name]
-> 1614 raise AttributeError("'{}' object has no attribute '{}'.format(
    1615     type(self).__name__, name))

AttributeError: 'CNN' object has no attribute 'out'

```

Evaluate the model on test data

We must call `model.eval()` to set dropout and batch normalization layers to evaluation mode before running inference.

model.train() tells your model that you are training the model. So effectively layers like dropout, batchnorm etc. which behave different on the train and test procedures know what is going on and hence can behave accordingly.

You can call either model.eval() or model.train(mode=False) to tell that you are testing the model.

```
In [ ]: def test():
        # Test the model
        cnn.eval()
        with torch.no_grad():
            correct = 0
            total = 0
            for images, labels in loaders['test']:
                test_output, last_layer = cnn(images)
                pred_y = torch.max(test_output, 1)[1].data.squeeze()
                accuracy = (pred_y == labels).sum().item() / float(labels.size(0))
            pass
        print('Test Accuracy of the model on the 10000 test images: %.2f' % accuracy)

        test()
```

```
In [ ]: sample = next(iter(loaders['test']))
        imgs, lbls = sample
```

```
In [ ]: actual_number = lbls[:10].numpy()
        actual_number
```

```
In [ ]: test_output, last_layer = cnn(imgs[:10])
        pred_y = torch.max(test_output, 1)[1].data.numpy().squeeze()
        print(f'Prediction number: {pred_y}')
        print(f'Actual number: {actual_number}')
```

```
In [ ]:
```

```
In [ ]:
```